

Symmetry-preserving Neural Functional Design for Density Functional Theory

William Chen

September 2, 2025

1 Introduction

1.1 Neural Functionals in DFT

Classical density functional theory (DFT) is a powerful tool in physics and chemistry for analyzing various behaviors of many-particle systems. However, the density functional itself is not easily obtained from analytical methods. Specifically, the excess free energy $F_{exc}[\rho]$, a functional of the space-dependent density, is difficult to find [1, 2].

With advancements made in machine learning, DFT research has focused on developing neural functionals to predict the one-body correlational values $c_1(\mathbf{r})$ [1-3]. Classically, the correlational values can be obtained by the following Euler-Lagrange equation with GCMC simulation data of equilibrium density $\rho(\mathbf{r})$, thermodynamic beta β , external potential $V_{ext}(\mathbf{r})$, and chemical potential μ [1].

$$c_1(\mathbf{r}) = \ln \rho(\mathbf{r}) + \beta V_{ext}(\mathbf{r}) - \beta \mu \quad (1)$$

The correlational relates directly to the free energy functional the following relationship derived through functional calculus.

$$c_1(\mathbf{r}; [p]) = -\frac{\delta \beta F_{exc}[\rho]}{\delta \rho(\mathbf{r})} \quad (2)$$

As such, recent research has aimed to develop neural networks to map $\rho(\mathbf{r}) \rightarrow c_1(\mathbf{r})$ given a full density profile. Utilizing a neural functional bypasses the analytical issues with deriving the free energy for complex systems.

1.2 DeePMD-Based Architecture

The neural functional analyzed in this report implements a DeePMD architecture [5] consisting of a feature and a description network. The feature network takes local density data within an R_c cutoff along with relative distance vectors for each neighbor. The output of the first network inscribes the detailed features of the density distribution, which is contracted and passed into the description network. The final output of this network provides a potential energy prediction for a point in space. The gradient of the output gives a force prediction that is used as the correlational value $c_1(\mathbf{r})$.

This goal of this implementation is to introduce the preservation of translational and rotational symmetry into a mesh-free neural functional. The DeePMD architecture achieves this by considering the sum of potential energies $E = \Sigma E_i$, which is symmetry-preserving by nature [5]. Additionally, for each point in space, the local coordinate frames are considered to ensure that the direction vectors are relative and not dependent on Cartesian coordinates.

1.3 Research Goals

This project aims to improve the original model architecture and training protocols to accelerate convergence while reducing loss values in the testing and validation stages. The updated implementation continues to utilize a DeePMD design to preserve important symmetries and focuses mainly on improving training efficiency.

2 Methods

The neural functional architecture was coded in Python utilizing the Jax, Flax, and Optax ML modules. Training and validation was conducted on a supercomputer cluster with a NVIDIA A100 GPU.

2.1 Dataset

The training data set was generated via GCMC simulation [4] and contains 101 snapshots of a periodic (80, 80, 80) grid of points. Training data was arbitrarily selected to include snapshots 0-79, with testing on snapshots 80-89 and validation on snapshots 90-100. One body direct correlational values were calculated according to the DFT Euler-Lagrange equation given constant values of inverse temperature β and chemical potential μ . Density $\rho(\mathbf{r})$ and external potential $V_{ext}(\mathbf{r})$ are positionally dependent [2].

2.2 Original Implementation

The model architecture was originally built with the following hidden layer specifications for the feature and description networks.

- Feature Network: [12, 24, 48]
- Describe Network: [64, 64, 64, 1]

The tensor output of the feature network is sliced at an arbitrary length to obtain left and right features. These two pieces are contracted by a partial dot product and passed into the description network. Further training specifications are listed below.

- Learning Rate Function: Exponential decay from 1e-5 to 1e-7 with a decay rate of 0.9
- Optimizer: Adam
- Training Scheme: select one permutation of points for all snapshots

For each training and testing epoch, the overarching scheme involves training on one permutation of points over all snapshots.

2.3 Updated Model Architecture

The basic functionalities of the updated neural functional involve a feature network and descriptor network, similar to the original model. For this report, the feature network is divided into two separate left- and right-feature networks with the following hidden layer neuron counts.

- Left Feature Network: [6, 6, 12]
- Right Feature Network: [12, 12, 24]
- Describe Network: [64, 64, 64, 1]

Instead of slicing a single feature tensor, the two networks individually output left and right features. The descriptor network and the feature contraction scheme remain the same.

2.4 Updated Training

The exponential decay learning rate function is updated to a cosine learning rate function with $\sim 5\%$ linear warm-up. Introducing a short warm-up is a common strategy in deep learning to adjust losses for larger training cycles.

- Learning Rate Function: Cosine decay from 6e-4 to 1e-8 with linear warm-up
- Optimizer: AdamW

The training scheme also changes with the updated model architecture. A mix of different training methods was tested for the model architecture described.

- Random sampling: for each snapshot, obtain a new permutation of random points.

- Cluster sampling: for each snapshot, randomly obtain a central point and its 26 neighboring lattice points.
- Mixed sampling: cluster training is sparsely intertwined with random sampling.

This final training schedule includes a mixed sampling procedure consisting of randomized and clustered data points. In the first 10,000 steps, clustered points are selected every 5 epochs. Clustered points are selected every 10 epochs afterwards to prevent the loss from spiking.

3 Results

This section details the architecture and training changes made in comparison to the original architecture and training procedures.

3.1 Model Optimizations

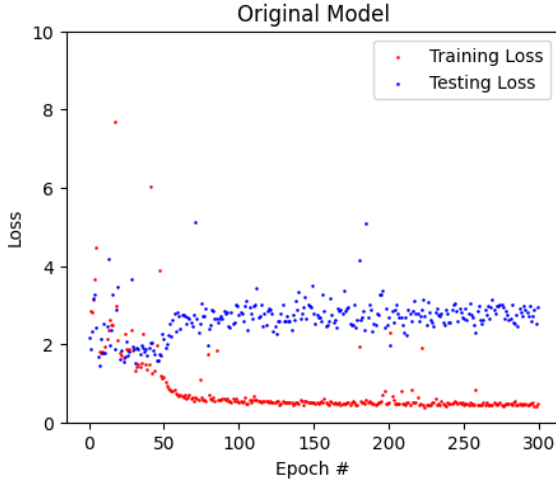


Figure 1: Loss curves for the original double network architecture following the original training scheme.

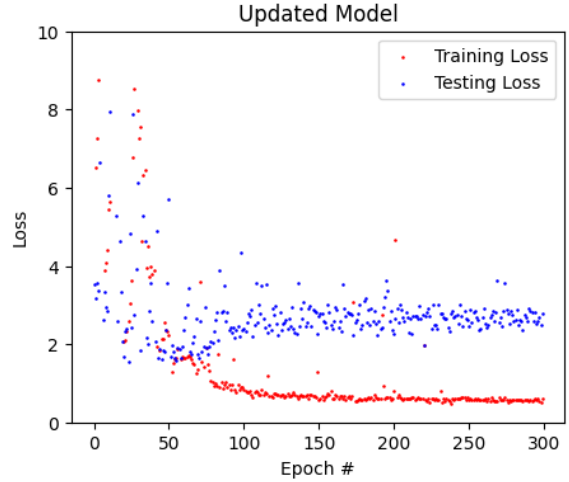


Figure 2: Loss curves for the updated network architecture following the original training scheme.

The training protocols for both model architectures result in standard loss curves, with a quick plateau of testing loss as convergence is reached. The original model displays convergence in approximately 30% less steps than the updated model. However, the updated model showed a lower mean test loss and normalized validation loss, 2.70 ± 0.26 and 0.0093 respectively compared to the original model's 2.78 ± 0.30 and 0.0099 .

3.2 Training Optimizations

Table 1 details the convergence statistics of each training method with the updated model architecture compared to the original model and training design. The updated training methods significantly improved the convergence time of the updated model, possibly due to the greater variety of data provided per snapshot.

Compared to the original iteration, the random sampling method showed a notable improvement in convergence time, but similar loss behaviors. Cluster sampling showed a worse convergence time and losses, but the loss variance after convergence was significantly lower. This marked an improvement in the frequency of loss spiking after convergence.

Figure 3 displays the updated model architecture trained with a mixed protocol of fully random and clustered sampling. The mixed sampling method integrates both cluster and random sampling to reflect stronger convergence in training, while also ensuring faster training. This iteration saw an improvement of $\sim 20\%$ in convergence time and testing and validation losses. In addition, the variances in testing and training were significantly reduced, per Table 1. However, it should be noted that the training epochs in which cluster sampling was used show relatively larger losses compared to those of the random sampling epochs. This is due to the rough integration of the two sampling methods, which can be improved to ensure a smoother training loss curve.

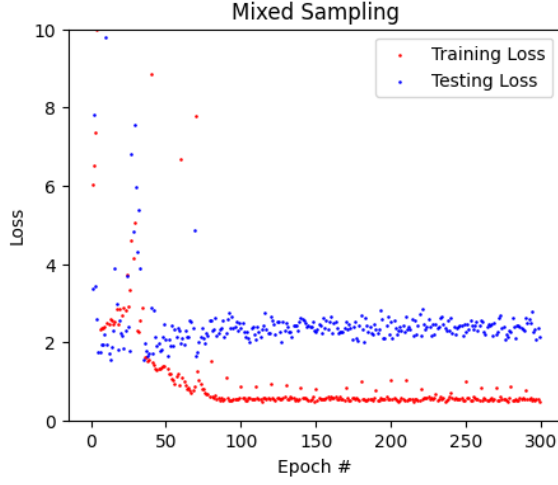


Figure 3: Loss curves of the updated model architecture following a mix of random and cluster sampling methods

Model Iteration	Convergence Steps	MSE Training Loss (Sum)	MSE Testing Loss (Sum)	MSE Validation Loss (Normalized)
Original	75,000	0.53 ± 0.20	2.78 ± 0.30	0.0099
Cluster	80,000	0.85 ± 0.14	2.78 ± 0.20	0.0093
Random	65,000	0.55 ± 0.17	2.76 ± 0.25	0.0094
Mixed	58,000	0.58 ± 0.10	2.38 ± 0.17	0.0080

Table 1: Displays summary of results for each model iteration. Note that training and testing loss are computed by summing individual loss values in each batch. Validation loss is normalized for a single point.

4 Conclusion

This report highlights the implementation of DeePMD-inspired neural functional networks for density functional theory. Overall, the updated model architecture and training protocols did show an improvement in convergence time and loss values.

In the future, a more complete training structure could smoothly integrate the random and cluster sampling methods in a more stochastic manner. This could further accelerate convergence while also improving the loss curves. There could also be improvements seen in using the original training regimen to familiarize the model with the subtle snapshot differences.

5 Acknowledgements

I would like to thank Professor Huan Lei and Liyao Lyu for their guidance and assistance on this project. I would also like to acknowledge Michigan State University’s HPCC for providing computational resources.

6 References

- [1.] Florian Sammüller, Hermann, S., De las Heras, D., & Schmidt, M. (2023). Neural functional theory for inhomogeneous fluids: Fundamentals and applications. *Proceedings of the National Academy of Sciences*, 120(50). <https://doi.org/10.1073/pnas.231248412>
- [2.] Florian Sammüller, Hermann, S., & Schmidt, M. (2024). Why neural functionals suit statistical mechanics. *Journal of Physics Condensed Matter*, 36(24), 243002–243002. <https://doi.org/10.1088/1361-648x/ad326f>
- [3.] Jacobus Dijkman, Marjolein Dijkstra, René van Roij, Welling, M., van, & Ensing, B. (2025). Learning Neural Free-Energy Functionals with Pair-Correlation Matching. *Physical Review Letters*, 134(5). <https://doi.org/10.1103/physrevlett.134.056103>
- [4.] Thompson, A. P., Aktulga, H. M., Berger, R., Bolintineanu, D. S., Brown, W. M., Crozier, P. S., in 't Veld, P. J., Kohlmeyer, A., Moore, S. G., Nguyen, T. D., Shan, R., Stevens, M. J., Tranchida, J., Trott, C., & Plimpton, S. J. (2022). LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*, 271, 108171. <https://doi.org/10.1016/j.cpc.2021.108171>
- [5.] Zhang, L., Han, J., Wang, H., Car, R., & E, W. (2018). Deep Potential Molecular Dynamics: A Scalable Model with the Accuracy of Quantum Mechanics. *Physical Review Letters*, 120(14). <https://doi.org/10.1103/physrevlett.120.143001>